HIGH-LEVEL DESIGN

RAG-Based Customer Support Assistant

using LangGraph & HITL

An intelligent customer support system using RAG, ChromaDB, Gemini API, and Human-in-the-Loop.

1. System Overview

Problem Definition

In many companies, customer support answers are already available in documents like FAQs or policy PDFs.
However, searching these documents manually takes time and is not efficient.

This can lead to slow responses and poor user experience.

Proposed Solution

To solve this problem, we built a **RAG-based Customer Support Assistant**.

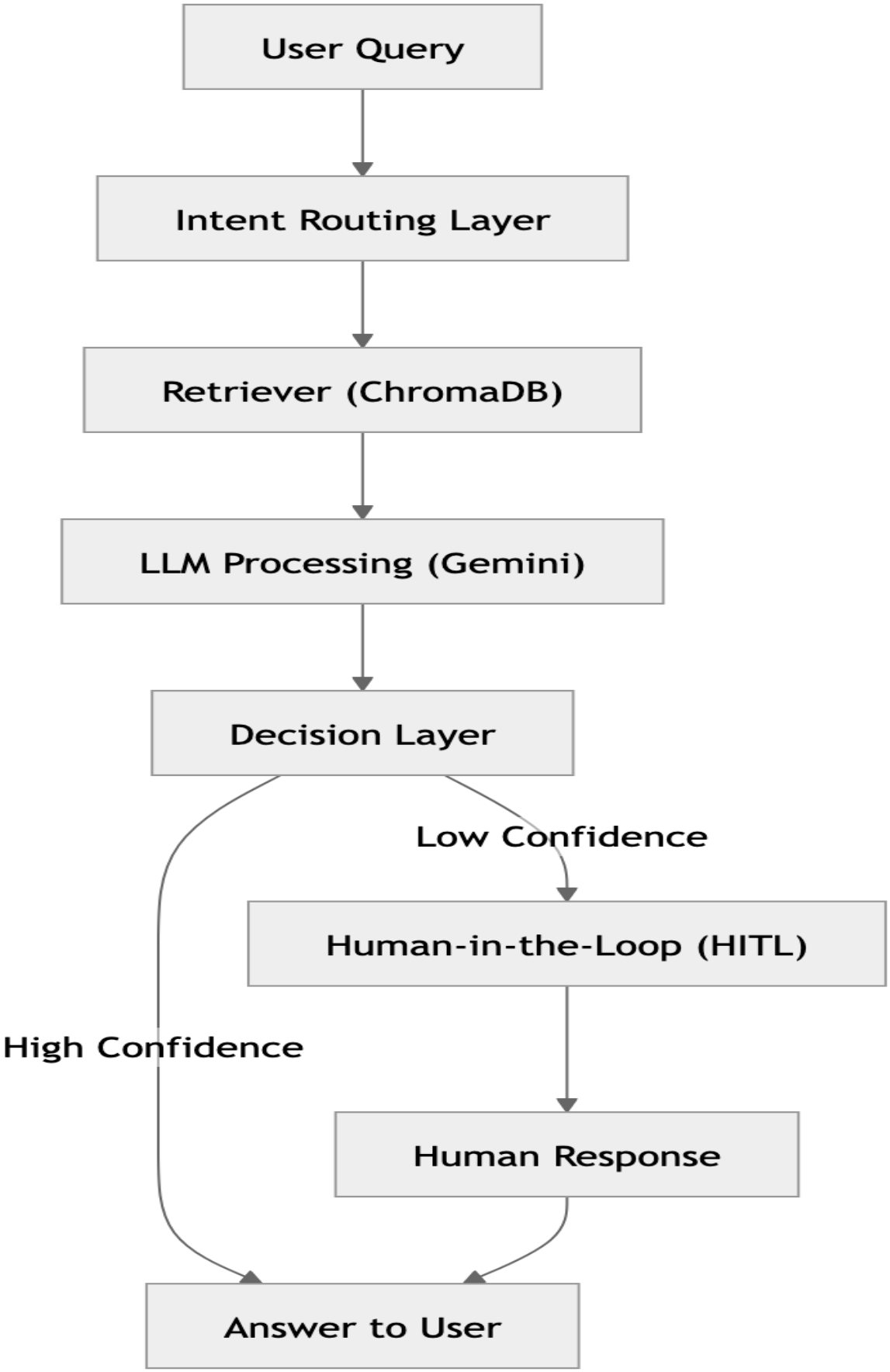
This system:

- Reads information from a PDF knowledge base
- Finds the most relevant content for a user query
- Generates an answer using an AI model
- Sends the query to a human if the system is not confident

This improves both speed and accuracy of customer support.

---

2. System Architecture (Explanation)


User Query
Intent Routing Layer
Retriever (ChromaDB)
LLM Processing (Gemini)
Decision Layer
Low Confidence
Human-in-the-Loop (HITL)
High Confidence
Human Response
Answer to User

---

3. System Components

Document Loader

This component loads the PDF file and extracts text from it.

Chunking

The text is divided into smaller parts so that it can be processed easily.

Embedding Model

Each text part is converted into numerical form (vector) so the system can understand similarity.

Vector Database (ChromaDB)

All vectors are stored in a database for fast searching.

Retriever

When a user asks a question, this component finds the most relevant text from the database.

LLM (Gemini)

The AI model uses the retrieved text to generate a human-like answer.

Workflow (LangGraph)

This controls how the system moves from one step to another.

Routing Layer

This decides whether:

- to give the answer directly
- or send it to a human

Human-in-the-Loop (HITL)

If the system is not confident, a human checks and gives the final answer.

---

4. Data Flow

1. User enters a query
2. Query is processed by LangGraph
3. Relevant data is retrieved from ChromaDB
4. AI generates an answer
5. System checks confidence
6. If high → return answer
7. If low → send to human
8. Final response is given to the user

---

5. Technology Choices

ChromaDB

Used for storing and searching data quickly.

LangGraph

Used to control the workflow and decision-making.

Gemini API

Used to generate answers using AI.

Python

Used to implement the entire system.

---

6. Scalability Considerations

Large Documents

The system splits large documents into smaller parts for better performance.

More Users

The system can be improved to handle more users by optimizing the database.

Speed

Using lightweight models helps in faster response time.

---

7. Summary

This system is not just a chatbot.
It is a smart AI system that:

- Retrieves correct information
- Generates accurate answers
- Uses human help when needed

This makes the system reliable and suitable for real-world use.